

ClaimTrek

A multiplayer GPS territory-capture running game · Mobile Systems (MobSys) 2026 · Dr.-Ing. David Sommer

Participants

Name	Matrikelnummer
Sahil Lakhani	320806
Rhitik Vishnu Nimbalkar	320993

What the app does

ClaimTrek turns a run into a territory game played on one shared map. You go for a run, and when your GPS track forms a **closed loop**, you claim the ground inside it. The claim is measured in real square metres, so a bigger loop is worth more. Running a loop over another player's ground **takes the overlap from them**: their plot shrinks, or disappears entirely if you leave them almost nothing. The map shows who currently owns what around you, and the leaderboard ranks players by the verified ground they hold.

Players sign in with **Google** before the app opens. Every claim is then published to **Cloud Firestore**, and each phone listens to the territory within about 15 km of it, so friends see each other's ground appear, and a steal reaches the player it was taken from within seconds. The phone's own SQLite database stays the place where claims are resolved, so the game never waits on the network: a claim made with no signal is saved at once and published when the signal returns.

A phone sitting in a car traces a perfect loop, so the app has to be hard to cheat. While you run it cross-checks your movement against the **accelerometer**, looking for the frequency signature of a running gait. A run that fails the check is still saved and drawn, but marked *unverified* and left off the leaderboard. The **barometer** records altitude and produces the elevation profile. Stopping a run needs the stop button to be **held**, so a bumped phone cannot end one by accident.

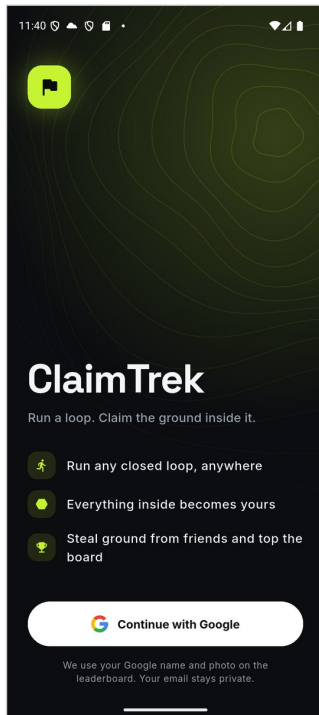
A second mode, **Treks**, queries the Overpass (OpenStreetMap) REST API for real named hiking routes within 5 km, filters them by length and draws the chosen one on the map.

How the exercise requirements are met

Requirement	In ClaimTrek
At least 3 Activities	Nine screens: Sign-in, Map, Run summary, Leaderboard, Run history, Run detail, Treks, Trek detail and Profile.
Data to the next Activity and back	The map hands the closed loop and its claim preview to Run summary; the Save / Discard decision is returned and only then is anything written to storage.
CustomView	Custom painters draw the elevation chart, the filling ring of the hold-to-stop button, the compass arrow to a trailhead and the contour background of the sign-in screen.
RecyclerView / list adapter	Recycling lists with custom card layouts for run history, the leaderboard and the trek list.
Two sensors	Barometer (altitude, climb, elevation profile) and accelerometer (gait anti-cheat). Also GPS, pedometer and compass.
Data storage	SQLite for territories, runs and the trail cache; key-value preferences for identity; Cloud Firestore for shared territory, accounts, runs and standings.
Networking / REST API	Overpass API for hiking routes (cached by map cell, as it is rate-limited); OpenStreetMap tiles; Firebase Authentication; live Firestore listeners.
Varied UI components	Floating tab bar with an animated indicator, filter chips, cards, text fields, progress bars, snackbars, a hold-to-confirm button and a colour picker.

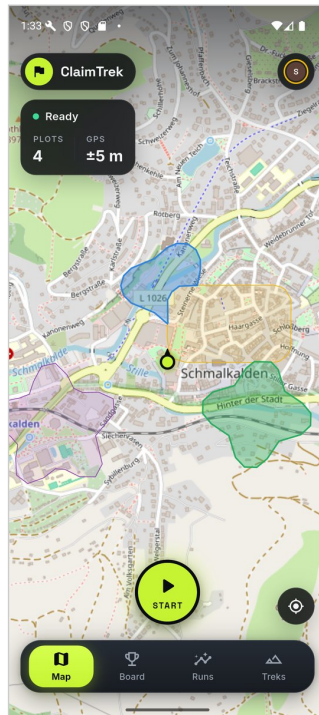
How the app looks

Screenshots taken on an Android emulator (Pixel 9a, API 36).



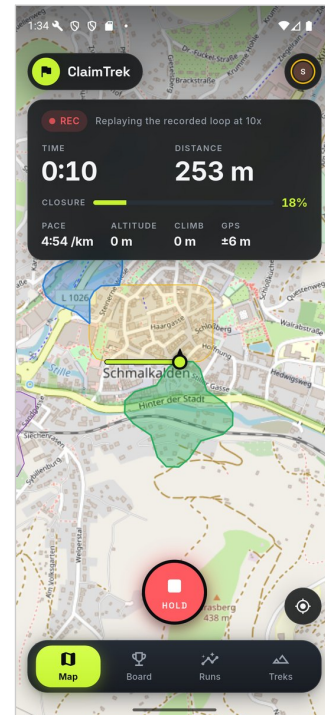
Sign in

The app opens here until the player signs in with Google; the account is what lets a steal reach the player it was taken from.



The map

Your ground and your rivals' in their own colours, a status strip, and the Start button. Territory within ~15 km is kept live from Firestore.



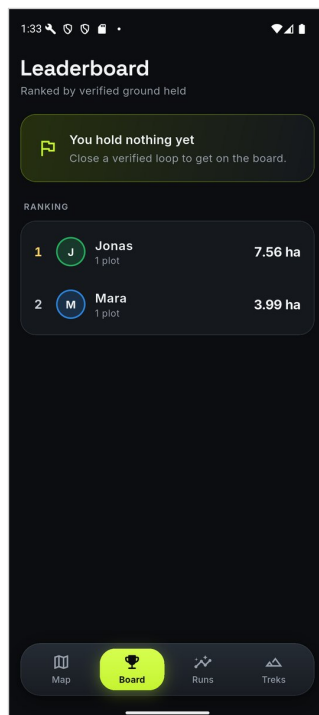
Tracking a run

Time, distance, loop-closure progress, pace, altitude and GPS accuracy. The stop button has to be held.



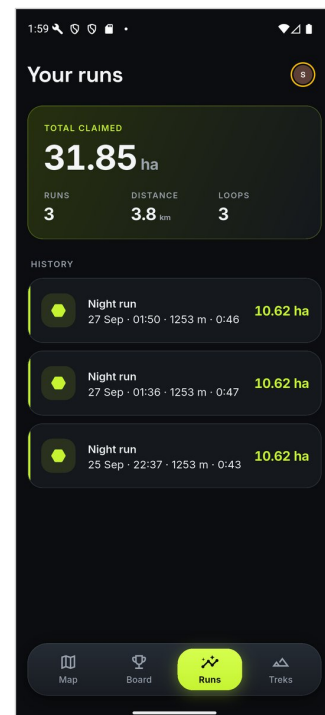
Run summary

Area claimed and taken from rivals, the barometric elevation profile (CustomView), and Save / Discard. Nothing is written until Save.



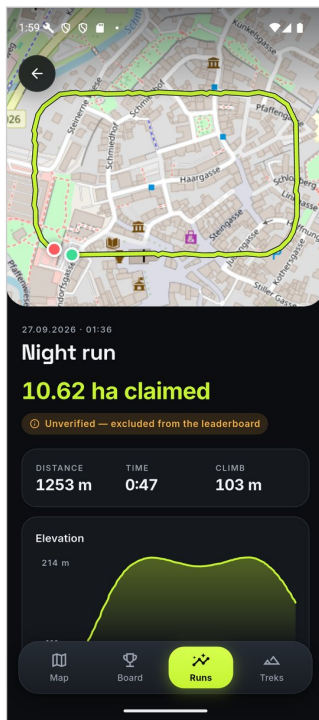
Leaderboard

Players ranked by verified ground held, with your own standing on top. Unverified runs are kept but do not count.



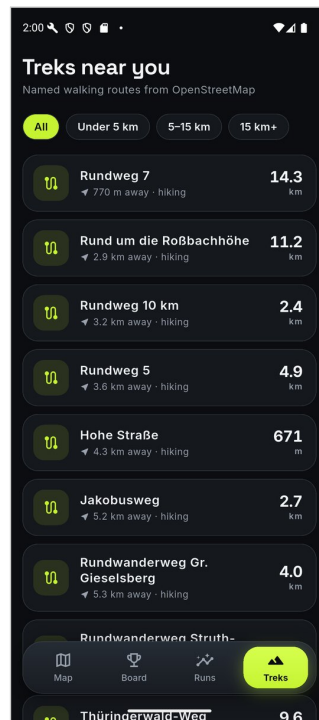
Run history

Every saved run from the local database, with lifetime totals; runs that took ground are marked in lime.



Run detail

A stored run reloaded from the database: the path redrawn with start and finish markers and the elevation profile rebuilt.



Treks — REST API

Named hiking routes near the runner from the Overpass API, cached locally and filterable by length.



Trek detail

The chosen route drawn on the map; a compass arrow (CustomView) points to the trailhead on phones with a compass.

Notes on the implementation

Framework used

The exercise specifies Android Studio with Kotlin. This project is written in **Flutter / Dart** and builds a native Android application (package `de.hsm.claimtrek`, minSdk 23, target API 36), which runs on the emulator and on a physical device. We are raising this openly rather than leaving it to be discovered, and are happy to discuss it.

Problems encountered

The geometry library did not work. Territory capture needs polygon overlay operations — intersection, union, difference. The library we chose first exposes all of these methods, but every one of them is an unimplemented stub that throws at run time. Because the error was caught and treated as "this territory is unaffected", stealing silently did nothing while appearing to succeed. We replaced it with a real clipping library and stopped wrapping geometry calls in catch-all handlers.

Degrees are not metres. Doing the geometry in latitude and longitude gives wrong areas, and two runs compared against each other shear apart. Every run is projected into a local planar metre frame about a reference point, stored per territory so the frame can be rebuilt later.

Two players can steal from the same plot at the same moment. If each phone simply uploaded its own version, the second upload would erase the first player's steal. The fix rests on one rule: once a territory exists it only ever shrinks — only its owner creates it, anyone may take ground from it, nobody may add ground to it in place. Two versions of a plot then merge by keeping only what both still hold, which gives the same answer in any order, so every phone converges on the same map. Uploads run inside a Firestore transaction, and the Firestore security rules enforce the rule on the server, so a modified app cannot grow, take over or revive someone else's plot.

A missing sensor must hide a feature, never break it. An absent sensor does not raise an error — its stream simply never delivers a value. Each sensor is probed at start-up and features behind a missing

one are hidden. Without this, a silent accelerometer would read as "movement with no gait", and every run on such a phone would be wrongly marked as cheating.

A plausible-looking GPS fix can still be wrong. A fix eighty metres from the truth can report six-metre accuracy and a walking pace, which is what signal bouncing off buildings produces. Only the distance from the previous fix, measured against the time between them, reveals it; a second filter does that, and gives up after repeated rejections so a runner who genuinely relocated is not locked out of their own track.

The game must not wait on the network. A Firestore write only completes once the server confirms it, which with no signal is never. Every change is therefore committed locally first and flagged for upload; a background pass publishes flagged changes and clears the flag only once the server has them, and every network step has a time limit, so saving a run is instant with or without a connection.

Note on the screenshots

The Android emulator's virtual barometer always reports exactly 1013.25 hPa, which the barometric formula maps to 0 m, so the elevation chart is flat on the emulator. For the screenshots the virtual sensor was driven through a realistic pressure curve so the chart shows genuine relief. The runs were driven by a recorded GPX loop replayed at ten times speed, which is also why they are marked unverified: a replay has no running gait. On real hardware the values come from the device's own sensors.

Testing

The project carries **248 automated tests** covering the geometry engine, loop detection, the anti-cheat analysis, GPS filtering, territory sharing between several simulated phones (including two players stealing from one plot at once, offline steals and malformed data from the network), storage, sign-in gating and the user interface. The full capture pipeline — GPS fixes, loop closure, claim, steal, save — is driven end to end from a recorded GPX track, so it can be tested indoors without running a real loop.